

UNIVERSITATEA POLITEHNICĂ DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DISERTAȚIE

Simularea expandării floricelelor de porumb

Vlad Marius-Cătălin

Coordonator științific:

Conf. dr. ing. Anca Morar

BUCUREȘTI

2022

UNIVERSITY POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DISSERTATION PROJECT

Popcorn expansion simulation

Vlad Marius-Cătălin

Thesis advisor:

Conf. dr. ing. Anca Morar

BUCHAREST

2022

CONTENTS

1	Introduction	1
2	Motivation	3
3	State of the art	4
3.1	Formal grammars	4
3.2	Particle systems	5
3.3	Manual swapping	6
4	Proposed Solution	7
4.1	Slicer	7
4.2	Puffer	8
4.3	Rigger	11
4.4	Collider adder	12
4.5	Animator	13
4.6	Jumper	13
4.7	Middle part expander	15
5	Implementation details	16
5.1	Asynchrony	17
6	Results	18
7	Conclusions	25
	Annex	26

SINOPSIS

Acest proiect prezintă o metodă de a genera și simula procedural expansiunea floricelelor de porumb. În prezent nu există studii similare în acest domeniu. Acest fenomen trebuie modelat și animat de mână. Algoritmul prezentat se concentrează pe tăierea și expandarea unui mesh 3D al unui bob de porumb în timp real. Videoclipuri cu expandări filmate cu încetinitorul au fost studiate pentru a crea o bază. De asemenea, studii anterioare despre expandarea floricelelor de porumb au fost examinate pentru o înțelegere mai bună a fenomenului. Progresul din mai multe iterații și rezultatele acestora sunt prezentate și comparate. Aplicația oferă o aproximare a întregului proces și un grad de controlabilitate asupra parametrilor aplicației.

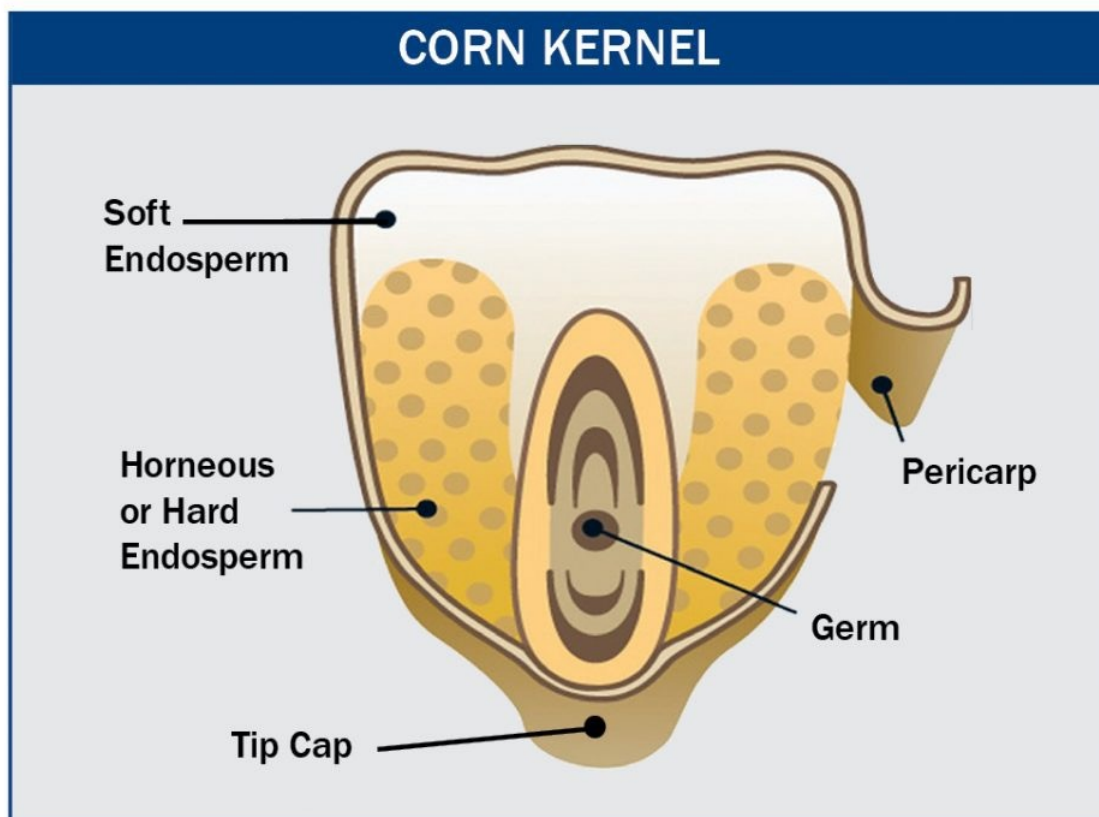
ABSTRACT

This project presents a method to procedurally generate and simulate the expansion of popcorn kernels. There has not been a similar study in the past. Such phenomenon must be manually modelled and animated. The algorithm presented in this paperwork focusses on cutting and expanding a 3D mesh of a kernel in real time. Slow motion videos were studied and used as baselines. Furthermore, previous studies about the expansion of popcorn kernels have been examined for a better understanding of the phenomenon. The progress over several iterations and their results are shown and compared. The application offers a rough estimation of the whole phenomenon and gives a certain amount of controllability over the parameters of the simulation.

1 INTRODUCTION

Corn, or maize, is a cereal grain cultivated and commercialised world wide. It has a variety of uses in different parts of industry (such as gastronomy and animal feed). One particular use is to create a common known snack called popcorn.

A special kind of species (*Zea mays*) is mostly used for cultivating popping corn kernels because its hard outer layer. When heated, the water inside the kernel evaporates and pressure increases. At around 177 °C [2] (Figure 2) the pericarp breaks and the starch inside the endosperm expands, forming the white part of the popcorn [5] (as seen in Figure 1).



Source: Adapted from Corn Refiners Association, 2011

Figure 1: Kernel's internal structure ¹

Not all species of corn can be used to make popcorn. Regular cereal kernels (such as wheat, rice, rye, etc. and other species of corn) have porous hulls which let the water vapours escape

¹<https://www.quora.com/Why-can-t-our-body-break-down-corn-yet-there-is-some-sort-of-corn-in-many-foods>

and prevent pressure build-up ². A study done by R.C. Hoseney et. al. done in 1983 [2] demonstrates that the percentage of kernels popped from a test group decreases with the amount of pericarp removed from individual kernel.

In another study done by Emmanuel Viroth et. al. [5] the cause of the 'pop' sound was researched. Initially, three main causes were conjectured: the crack of the pericarp, the jump of the kernel or the release of the build-up steam. The phenomenon was recorded several times at 2000 frames per second with both video and audio streams. It was found that the sound is produced 100ms after the outer hull is broken. They concluded the sound is caused by the release of the water vapours, similar to the sound heard when opening a bottle of champagne.

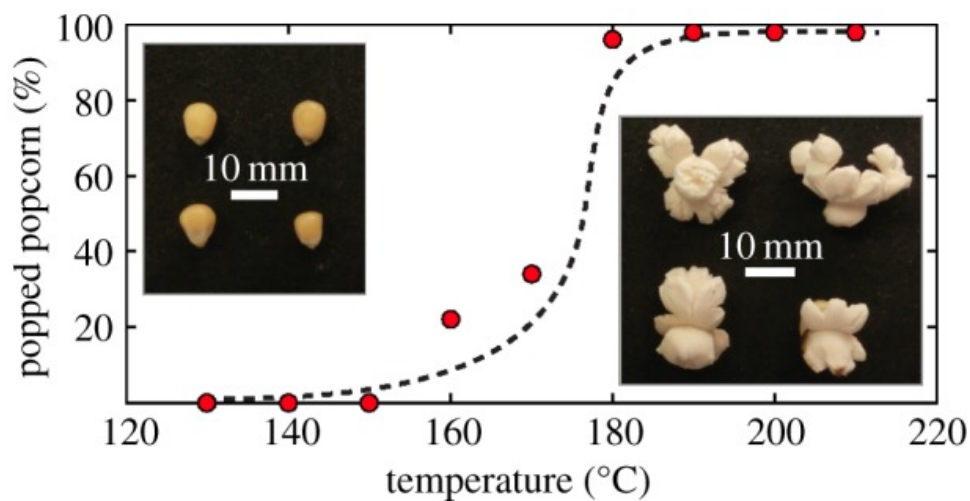


Figure 2: Percentage of expanded kernels in an oven when the temperatures rises gradually [5].

²<https://indianapublicmedia.org/amomentofscience/why-popcorn-pops-grains.php>

2 MOTIVATION

As technology evolved, the demand for realism in films and video games had rapidly increased [3]. Special effects and high levels of detail are in high demand. Nowadays, the majority of such content is done manually by skilled individuals. Such approach ensures a high level of control of the final product, but it is time consuming, hard to modify once generated and not very suitable for environments that are constantly changing [4].

Procedural generation is a process of automatically generating content. This method is less time consuming [3] and the generated content can even be superior to manually crafted assets [4]. In practice, procedural generation is highly used for environment creation such as landscapes, cities, forests, etc.

At the writing of this document, no other method of procedurally generating popcorn has been published. Such phenomenon is simulated by rapidly changing a manually crafted mesh of a popcorn kernel with another manually crafted mesh of an already expanded kernel. This approach gives a convincing effect since the popping happens too fast to be observed with the naked eye, but fails when the simulation needs to be rendered in slow motion.

The purpose of this project is to create a procedural algorithm which eliminates a high portion of manual labour when simulating the expansion of popcorn kernels.

3 STATE OF THE ART

In graphics and video games, procedural generations has been used since 30 years ago, especially in creating 3D models for vegetation [3]. Since manually crafting foliage and trees is very time consuming, automated processes have been perfected to generate realistic plants with minimum user interaction.

3.1 Formal grammars

Formal grammars are widely used because of their recursive form and simplicity [4]. It is formally represented as a tuple with the following components:

$$G = (N, \Sigma, P, S)$$

- N a finite set of non-terminal symbols (which can be replaced)
- Σ a finite set of terminal symbols (which can not be replaced)
- P a finite set of production rules (which define how non-terminal symbols can be replaced with terminal or non-terminal symbols)
- S the starting symbol ($S \in N$)

A simple example of generating a 2D plant is presented on Wikipedia ¹:

$$\begin{aligned} N &= \{X, F\} \\ \Sigma &= \{+, -, [,]\} \\ P &= \{(X \rightarrow F + [[X] - X] - F[-FX] + X), (F \rightarrow FF)\} \\ S &= X \end{aligned}$$

Where

F = draw forwards

$+$ = turn anticlockwise 25°

$-$ = turn clockwise 25°

$[$ = save the current position and angle

$]$ = replace the current position and angle with the previous ones

X is used to control the evolution of the curve

A generated plant with this grammar can be seen in Figure 3.

SpeedTree ² is a well-known programme which generates trees using formal grammars. It was

¹https://en.wikipedia.org/wiki/L-system#Example_7:_Fractal_plant

²<https://store.speedtree.com/>

used in the making of films such as Avatar and The Great Gatsby ³ and was incorporated in several popular game engines such as Unity and Unreal Engine 4.



Figure 3: Plant generated with formal grammars and fractals ⁴

3.2 Particle systems

In this approach, the apical growth of plants is used [1]. Meristem is a type of tissue which helps plants grow. It is found inside the tips of roots, branches and buds ⁵. This type of tissue contains undifferentiated cells that can continue to divide and differentiate (become regular cells) ⁶. The plant grows by extending its roots and stem and by transforming its buds in branches, leaves and flowers.

To generate a plant with this method, a main and alive particle (also called seed) is placed in the scene. The algorithm is iterative. Each step, all alive particles can spawn other alive particles and then die. Branches are created when a particle spawns more than one particle. The process stops when all particles are dead. This approach gives more control of the

³<https://www.livemint.com/Consumer/Jh1N22EQW0zbwbjcU8jORI/Hollywoods-movie-tech-wizards-honoured-by-Oscars-organizers.html>

⁴https://en.wikipedia.org/wiki/L-system#Example_7:_Fractal_plant

⁵<https://courses.lumenlearning.com/ivytech-bio1-1/chapter/how-plants-grow/>

⁶<https://pediaa.com/what-is-the-difference-between-differentiated-and-undifferentiated-cells/>

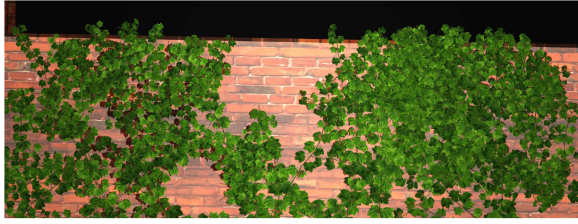


Figure 4: Plant generated with a formal grammar [4]

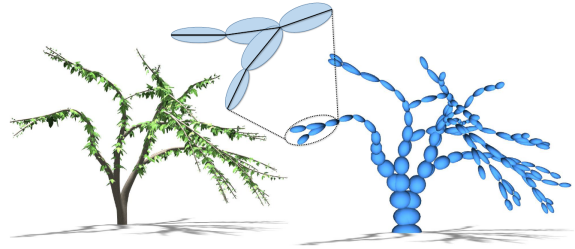


Figure 5: Plant generated with a particle system [1]

outcome and can be easily modified. Plants generated using grammars must be completely regenerated on every modification. A comparison between plants generated using these two methods can be seen in Figure 4 and 5.

Thomas Luft's Ivy Generator ⁷ is a popular stand-alone application which uses particle systems. When the plant grows, it takes gravity, collisions and light sources into consideration. The program is free and open-source. The algorithm was ported as extensions for several graphics programs, such as Ivy ⁸ for 3DS Max and hedera ⁹ for Unity Engine.

3.3 Manual swapping

The expansion of a popcorn kernel is very fast and the intermediate states cannot be easily seen with the naked eye. A good approximation of this phenomenon is to swap the 3D model of the original kernel with an already expanded one. Other effects such as adding a jump force can be added to enhance realism.



Figure 6: 3D models of popcorn kernels ¹⁰



Figure 7: Swapping kernel meshes with already expanded kernels ¹⁰

⁷<http://www.ivy-generator.com/>

⁸<http://www.evvisual.com/ivy/>

⁹<https://github.com/radiatoryang/hedera>

¹⁰<https://www.youtube.com/watch?v=DwQRQKxgtqo>

4 PROPOSED SOLUTION

Several videos of slow motion expansions have studied and a common pattern was observed. When the pericarp explodes, a small portion of the white expanded starch stays attached to it. From the point of explosion, the portions start to roll backwards and form a shape similar to the letter C ¹. Also, the most part of the starch stays in the middle and starts to grow in volume.

The main module will simulate the behaviour of these deductions (Figure 8). It will take a 3D mesh of a popcorn kernel as input, and will create an expanding kernel mesh. Each component has its unique purpose and will be described in the following sections.

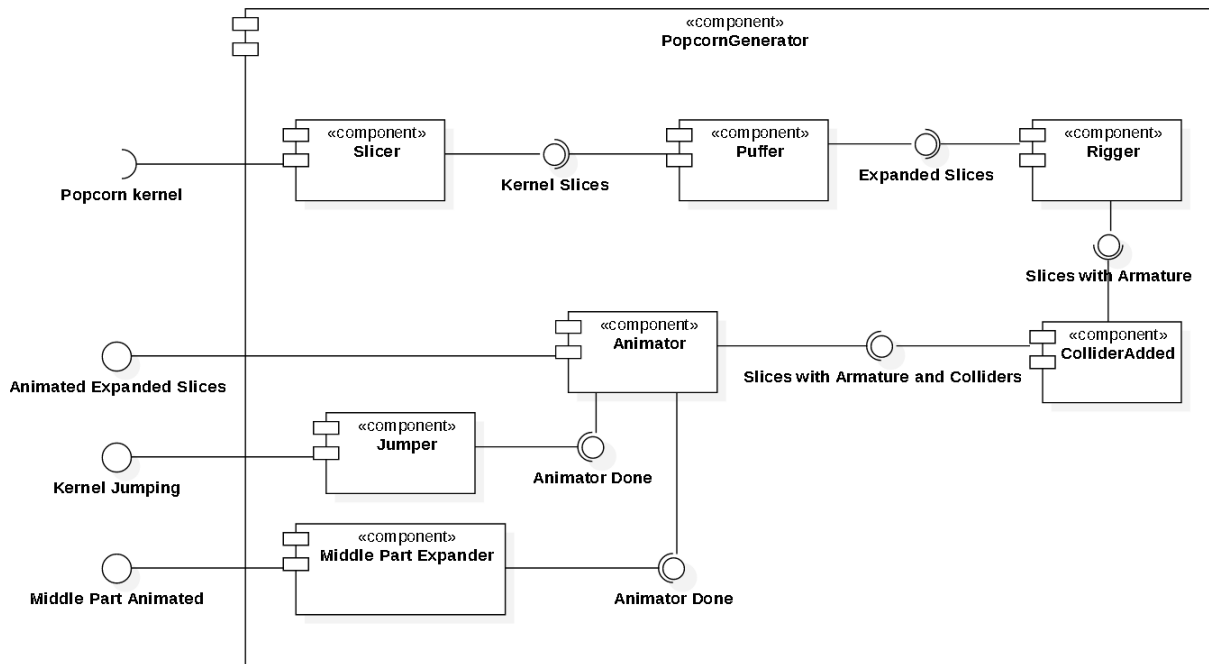


Figure 8: Modules of the application

4.1 Slicer

Slicer takes the 3D mesh of the original kernel and cuts it in slices. Planes are used to slice the triangular mesh since it is not computationally heavy to calculate the intersection between a plane and a triangle.

¹<https://www.youtube.com/watch?v=FSZd33awqQk>

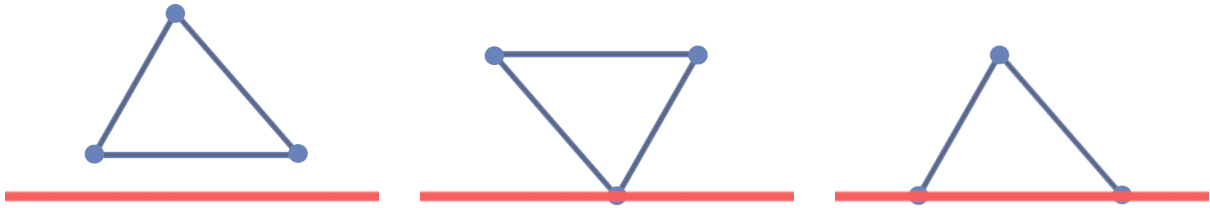


Figure 9: Cases where a triangle is **not** cut by the plane



Figure 10: Cases where a triangle is cut by the plane

For each triangle of the mesh, it is decided if it intersects a plane or not. Based on this information, the triangle can be cut or not. It will not be cut (Figure 9) when:

- All 3 vertices points are on the same part of the plane
- 1 vertex intersects the plane and the other 2 are on the same side
- 1 edge of the triangle is on the plane

Triangle will be cut (Figure 10) when:

- 1 vertex is on the plane and the other 2 are on different sides of the plane
- 1 vertex is on one side of the plane and the other 2 are on the other side

After cutting iterating through all triangles, cutting those that intersect and separating them, two different meshes are created (one with the triangles from each side of the plane). The resulting meshes can be also cut by another cutting plane to create more slices. A visual representation of the algorithm can be seen in Figures 11 - 13.

4.2 Puffer

After the slices have been generated, new sub-meshes for the white part are constructed. The first step is to determine the borders of the cut slices. In the previous stage, new vertices have been generated to separated the resulting slices. Those vertices are known to be situated on the cutting plane and actually represent the border of the mesh. In this stage, those vertices are used to create two border lines (Figure 14). Each line of vertices correspond to a cut done by a single plane.

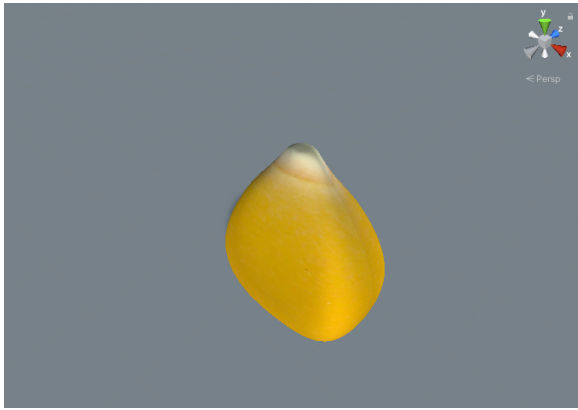


Figure 11: Popcorn kernel

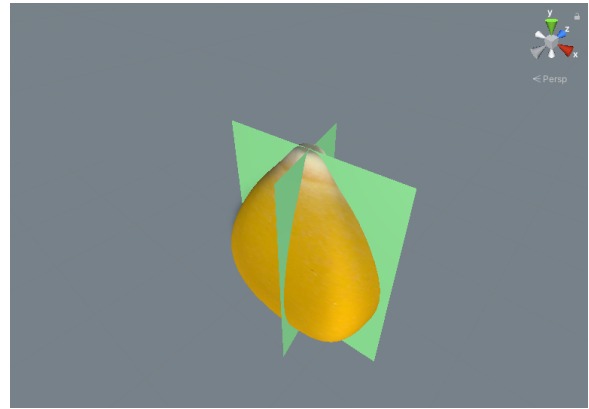


Figure 12: Popcorn kernel and 2 cutting planes

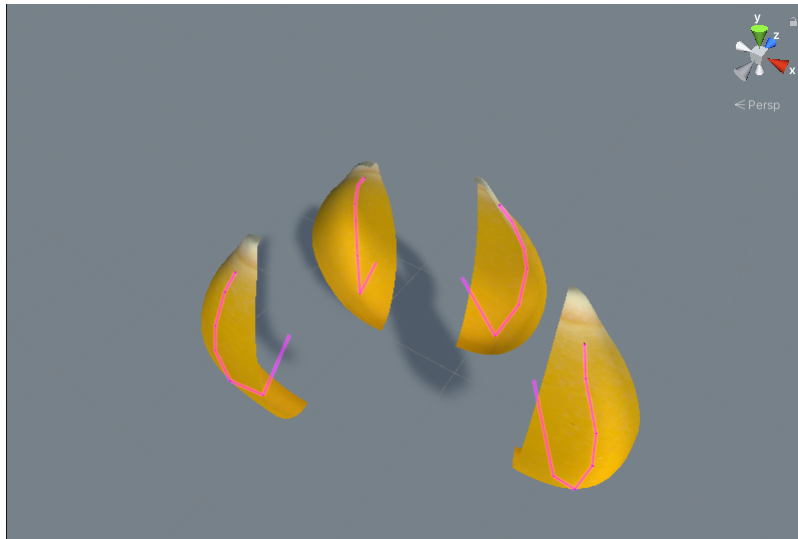


Figure 13: Resulting slices

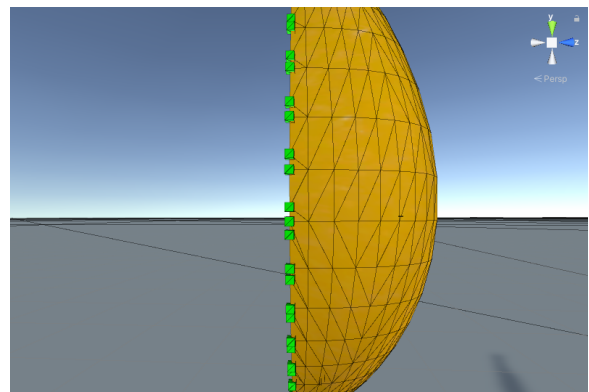
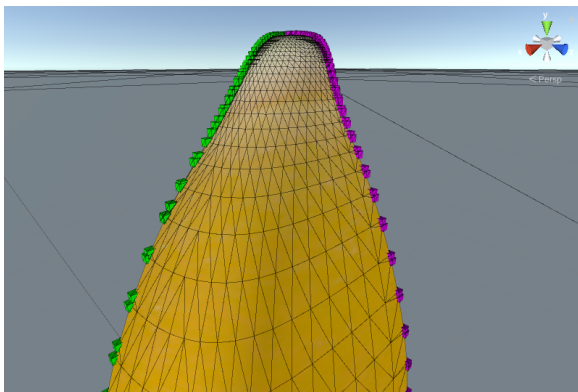


Figure 14: Border vertices of a slice

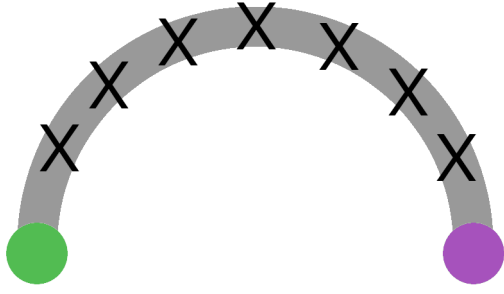


Figure 15: Generation of vertices on a semi-ellipse

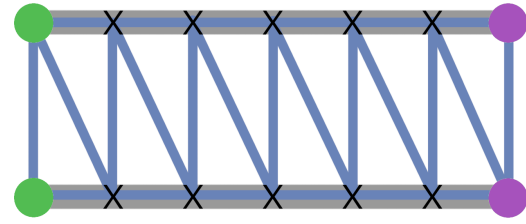


Figure 16: Generation of triangles between 2 semi-ellipses

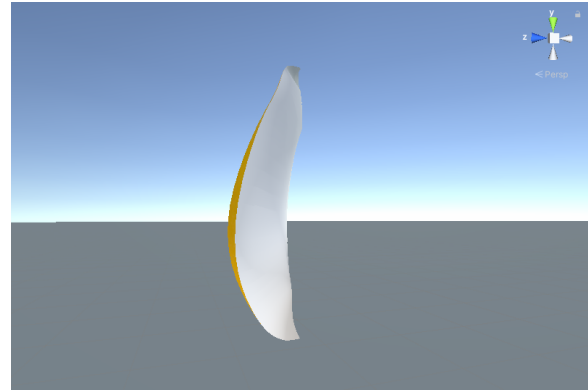
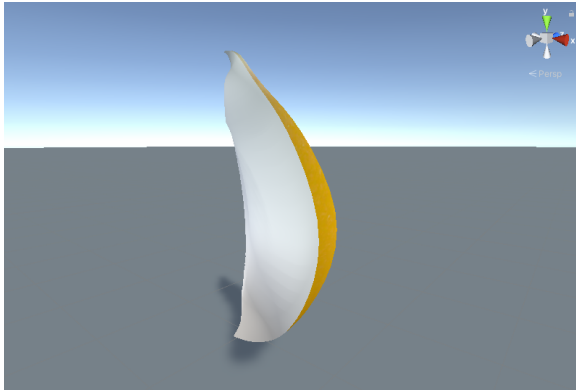


Figure 17: Expanded slice

The first step is to generate vertices for a new sub-mesh. The algorithm iterates through the border vertices of each lines, from top to bottom, and generates points on a semi-ellipse (Figure 15). This gives the impression that the puffed part inflated. The number of points generated on each semi-ellipse is the same for all slices to keep a consistent level of detail.

To generate triangles, vertices from 2 consecutive curves are used as seen in 16. For the first and last curves, a fan triangulation ² is used to fill the gaps. The puff for one slice can be seen in Figure 17.

A fast and improvised algorithm was used to generate the UV coordinates of the puff mesh. The centre of gravity and mean surface normal are calculated in the first step. Then, all vertices are projected on the plane defined by the point and normal determined previously. The plane is rotated, translated and scaled so that it coincides with the UV coordinate space. A final downscale to $(0.05, 0.05) - (0.95, 0.95)$ is done to prevent possible floating point approximation errors.

The part in the middle of the expanded kernel is not procedurally generated. A pool of pre-made meshes is used. When the kernel expands, a random one from the pool is instantiated and is given a growing animation (upwards translation and up-scaling).

²https://en.wikipedia.org/wiki/Fan_triangulation

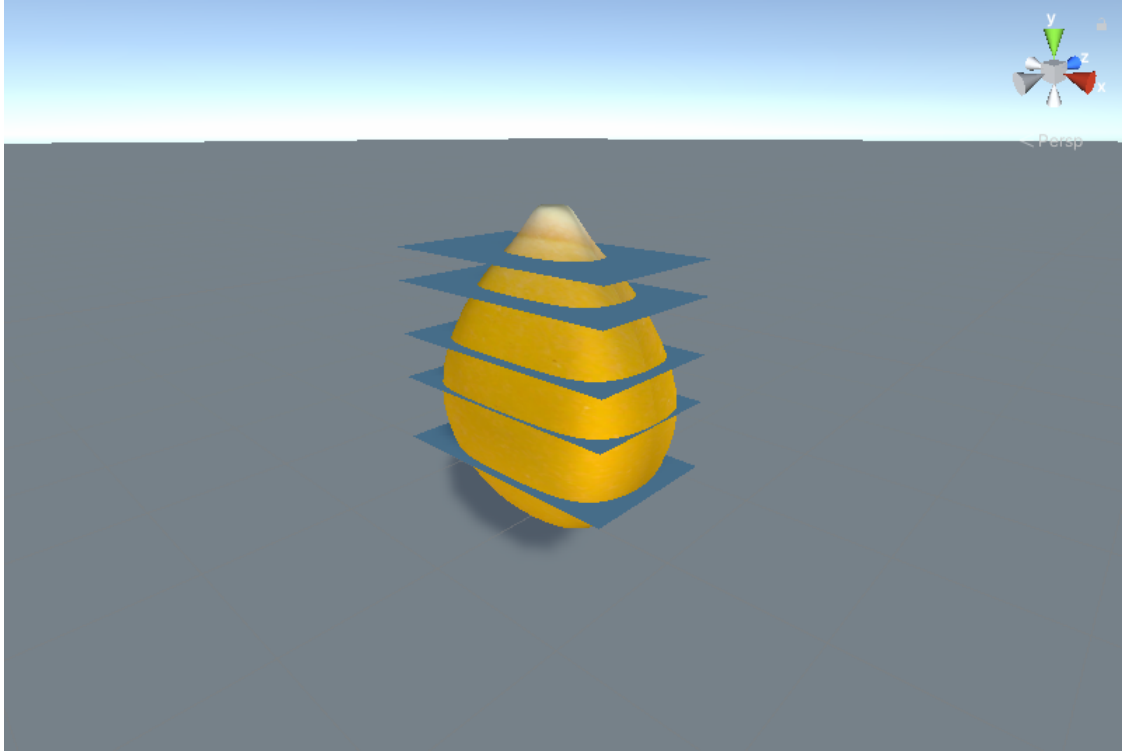


Figure 18: Popcorn kernel and rigging planes

4.3 Rigger

To animate the puffed slices, an armature needs to be generated. Since the slice needs to be animated to bend backwards, a simple and linear bone hierarchy is used. To separate the zones of action of each bone, rigging planes are used (Figure 18). These will delimit special zones where a bone will be generated.

For each slice, all the vertices are split by the rigging zones. For each zone two important points are calculated, the centre of gravity of the whole slice and the centre of gravity of only the kernel vertices (without the puff vertices). The position of the bone will be the middle of the segment formed by the two points calculated previously (Figure 19). The tests showed that this position is optimal and creates less visible creases when the slices bend backwards.

The vertices follow bones' positions, rotations and scales. A way to define such behaviour is using bone weights. This way, each vertex will interpolate between multiple bones' transforms to create smoother transitions between close rigging zones. In this algorithm, the inverse distance to each bone is used. For zones which have exactly 2 adjacent zones, the weights are as follows:

$$m = \frac{1}{d_0} + \frac{1}{d_1} + \frac{1}{d_2} \quad (1)$$

$$w_0 = \frac{1}{md_0} \quad (2)$$

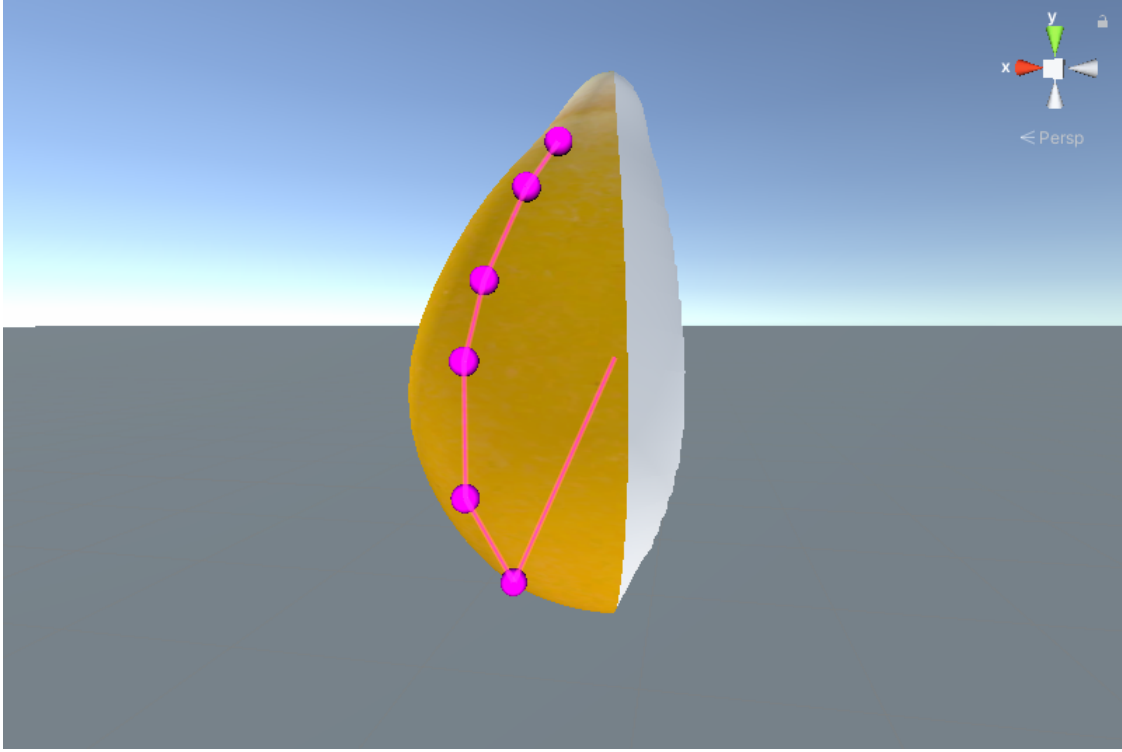


Figure 19: Slice with armature. Spheres represent the bones and the lines their relation. The bottom bone is the root bone

$$w_1 = \frac{1}{md_1} \quad (3)$$

$$w_2 = \frac{1}{md_2} \quad (4)$$

where d is the euclidean distance between the vertex and the bone, w is the bone's weight, 0 is the index of the current zone and 1 and 2 are the indices of the adjacent zones.

For zones that have only one adjacent zone (the top and bottom ones), the vertices follow the zone's bone by 0.75% and the adjacent bone by 0.25%.

Creases are still visible if the camera is close to the slice, but indistinguishable when looking from a distance of several decimetres.

4.4 Collider adder

To enhance the realism of the application, colliders and rigid bodies are given to the expanded kernel. Since the purpose of this program is to simulate hundreds of simultaneous expansions and collision detections and reactions are computationally expensive, compromises have to be made. The entire mesh of the expanded kernel can be used as collision object, but it has a high number of vertices and triangles. Such approximation is very inefficient.

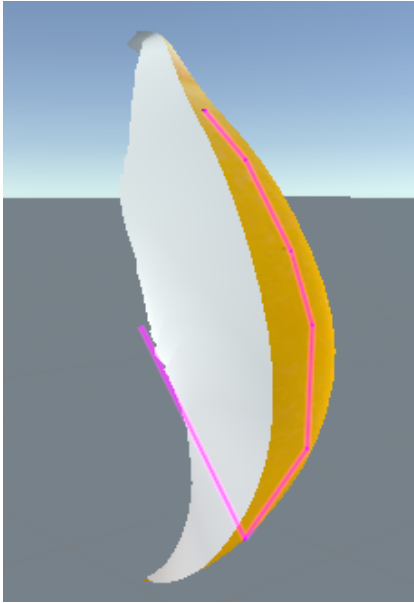


Figure 20: Slice

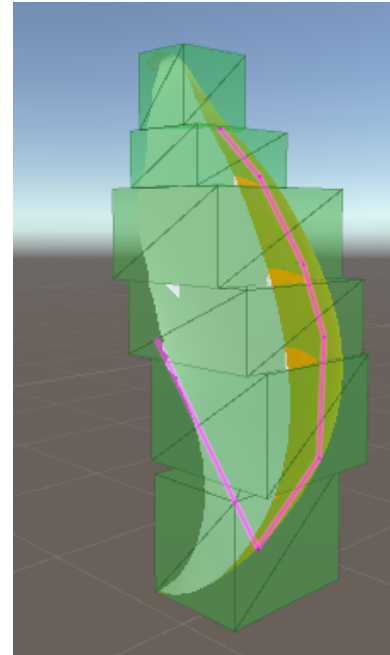


Figure 21: Slice with attached OBB **colliders**

For each slice, the rigging zones are used to calculate an object oriented bounding box ³ (OBB). Each zone gets an OBB which is attached to its bone. This method ensures the collision of the slices will be function correctly when the slices bend backwards (Figure 20 and 21). The middle part receives a simplified version of the mesh as a collider (Figure 22 and 23).

4.5 Animator

Based on the observation of slow motion videos of expanding popcorn kernels, the small slices tend to bend backwards. This component's purpose is to simulate such behaviour. The armature generated in the Rigger stage is used here. The animation consists of rotating the bones backwards. The rotational direction is given by the direction towards the previous bone in the hierarchy and one of its perpendiculars (Figure 24). The zone at the bottom of the slice will not rotate since it does not have a previous zone.

4.6 Jumper

To simulate the jump of the kernel when it explodes, a force is applied to the rigid body component. A random point is generated on the spherical cap of a steradian angle around

³<https://subscription.packtpub.com/book/game-development/9781787123663/7/ch071v11sec70/oriented-bounding-box>

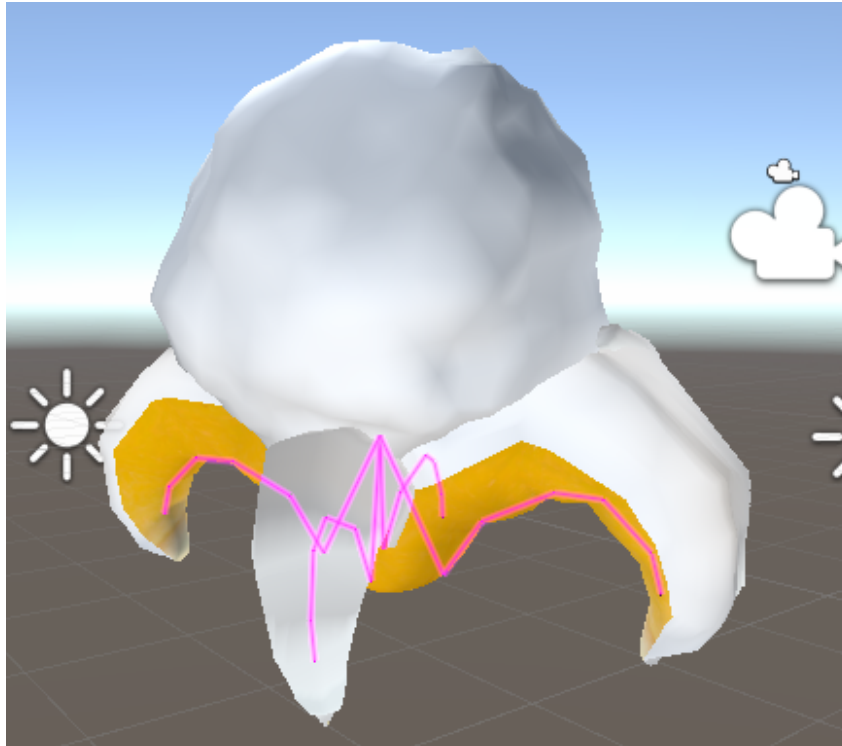


Figure 22: Expanded kernel

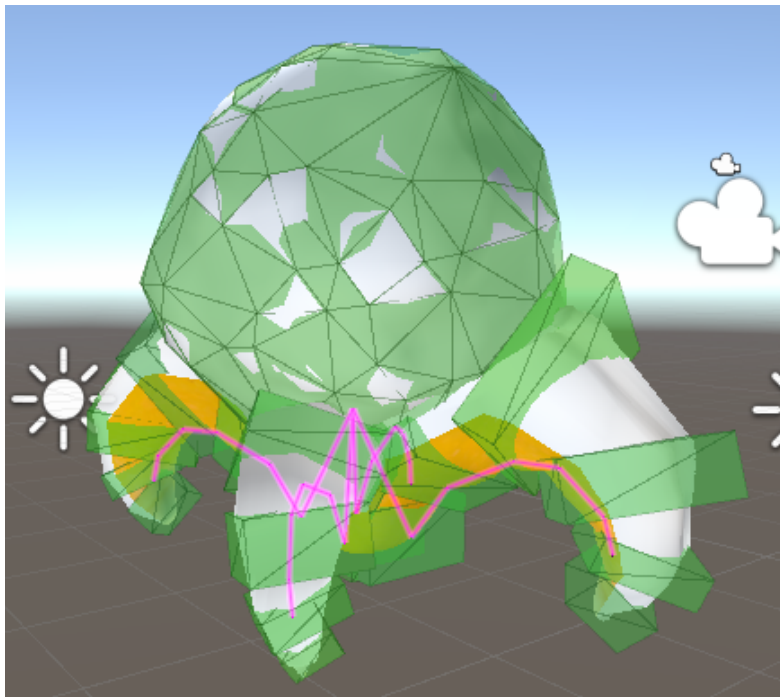


Figure 23: Expanded kernel with attached colliders



Figure 24: Bones rotation

the upwards axis (Figure 25). The direction of the launch is given by the centre of the kernel and the generated point. The values of the force and angle are customisable. A small torque in a random direction is also added to increase realism.

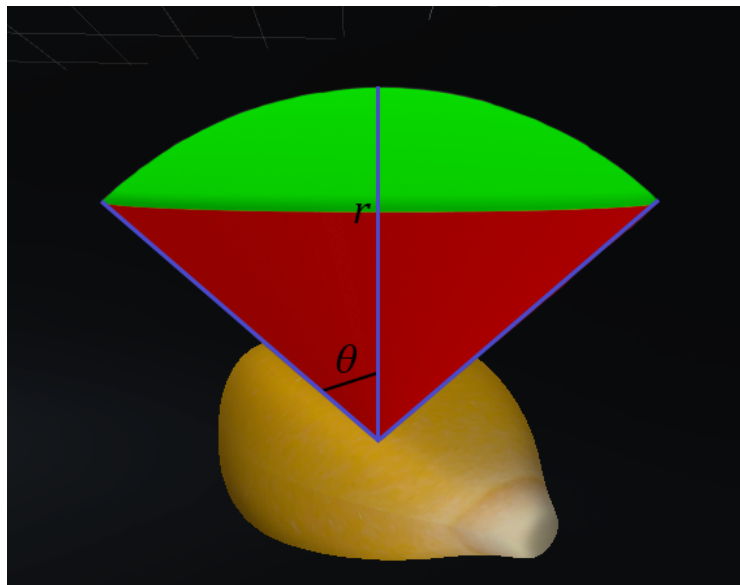


Figure 25: Force applied to the kernel, r force value, θ maximum force angle

4.7 Middle part expander

After the animations to the slices are added, the centre puff part also receives animations. The scale and positions of the mesh are altered so that it simulates a small growth in size.

5 IMPLEMENTATION DETAILS

Unity Engine ¹ was used to develop this application. It offers an easy to use C# API and graphical interface to access and modify 3D objects in real time. The engine is written in C++. This creates a significant time and memory overhead when writing scripts in C# since data must be copied and adapted from C++.

The cutting algorithm used in the Slicer component was inspired from Ezy-Slice ². Initially, the cut was in-place, the original mesh was cut, no additional meshes were generated. This put less pressure on the C#'s garbage collector since no additional memory was allocated on the heap, but the overall mesh was harder to process. After the mesh was cut, there was no information about the slices. A non-directed graph from the mesh was created and flood fill ³ was used to determine each slice. This implementation was slow and did not give the desired results. The 3D model used was taken from SketchFab ⁴. It has a seam so that a 2D texture can be mapped to a 3D model. This made the vertices to be duplicated on this seam. The flood fill recognised this seam as a cut. This approach was replaced with the current one.

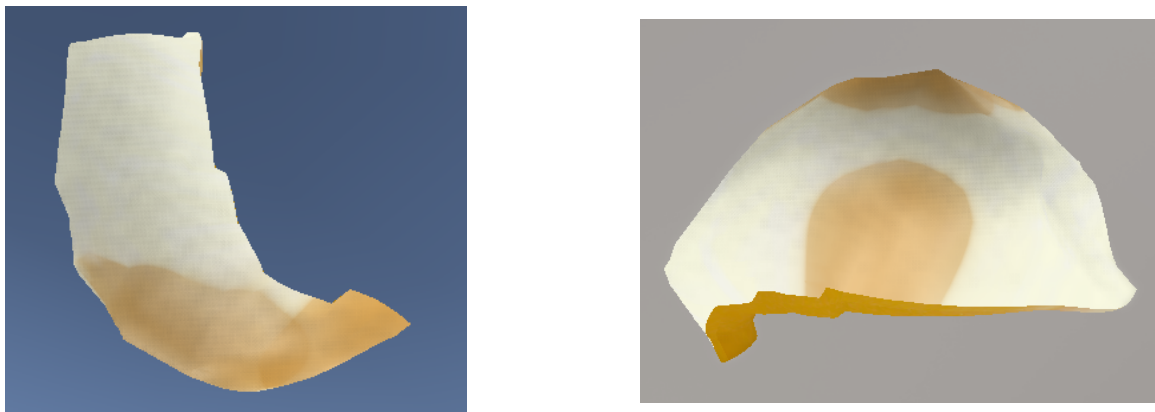


Figure 26: Burnt slices

The shader of the puffed part can also simulate burn points. In reality, when the white expanded starch touches a hot surface (i.e. a hot frying pan), its colour starts to darken. This behaviour is simulated with the Unity's collision system ⁵. When an OBB of a slice touches a collider marked as 'hot', a short burn animation inside the shader is played. The original texture is blended with a burn texture around the contact points (Figure 26). The animation consists of enlarging a circle mask between the two textures.

¹<https://unity.com/>

²<https://github.com/DavidArayan/ezy-slice>

³https://en.wikipedia.org/wiki/Flood_fill

⁴<https://sketchfab.com/3d-models/corn-kernel-284db06b8d094a6382c62913083335d7>

⁵<https://docs.unity3d.com/ScriptReference/Collision.html>

5.1 Asynchrony

The initial implementation was synchronous (all stages were executed in a single frame, as seen in Figure 27). This caused a massive spike in the frame-rate of the application since the entire process took $180 - 200ms$. To improve the performance and not affect the number of frames per seconds (FPS), the process was split in mini-stages which were executed in different frames (to a asynchronous model, as seen in Figure 28). Initially, a mini-stage took between 0.5 and $10ms$, but was later improved to last around $1 - 2ms$. Some mini-stages could not be split since atomic operations (such as memory allocation) were execute and can run up to $8ms$.

This allowed the execution of multiple expansions in parallel without a significant drop in FPS. The first attempt was to run the simulation of all expansions in parallel (Figure 29). This caused a significant increase in the frame time, especially for a high number of expansions. The second attempt was to delay the start of each expansion (Figure 30). This helped the performance and gave a randomness effect since the expansions do not happen at the same time in reality. However, the simulation time was slightly increased and did not help the performance for a large number of expansions. The final version contains a better management of the expansions. A configurable number of processes can be run in each frame. When the first expansion finishes, the number of processes executed per frame is decreased to compensate for the overhead generated by the engine.



Figure 27: Synchronous execution of an [expansion](#)



Figure 28: Asynchronous execution of an [expansion](#)



Figure 29: Asynchronous execution of [two expansions](#) started in the same frame

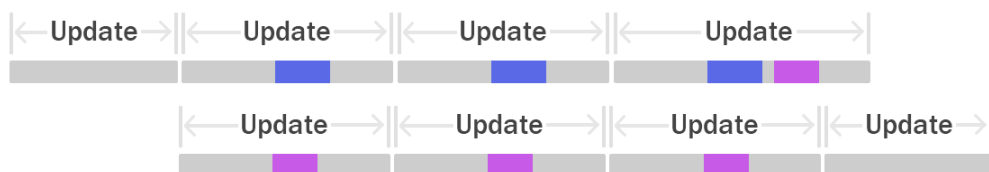


Figure 30: Asynchronous execution of [two expansions](#) started in different frames

6 RESULTS

A scene with a single kernel was created to test the performance of a single expansion. Initially, the kernel could be cut by 2 or 3 planes. This method was later abandoned since the algorithm was very stable and could cause several issues (i.e. puff was not generated for a slice, borders were not found correctly). The execution time was recorded in Figure 31 and 32. A significant performance cost is introduced by adding a cutting plane (from $180 - 200ms$ to $245 - 265ms$ when executing synchronously).

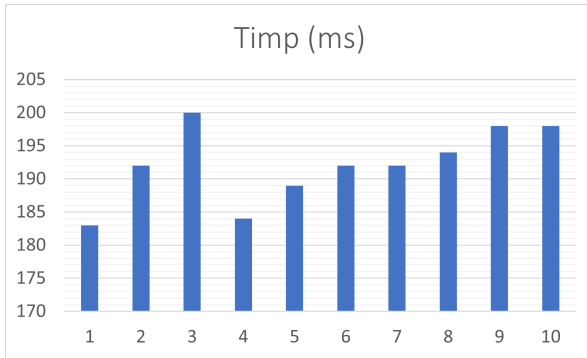


Figure 31: Execution time when 2 cutting planes are used

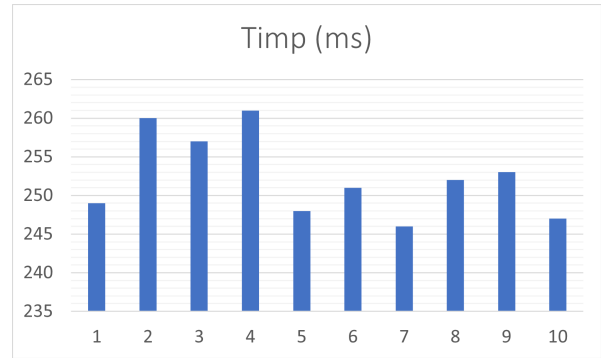


Figure 32: Execution time when 3 cutting planes are used

After switching to the asynchronous model, the total time of executing a single expansion increased significantly (to $300 - 350ms$ for the whole process). This was caused by the overhead generated by stopping the execution in a frame and resuming in another one.

A new test scene was introduced. The user can spawn 9 kernels when pressing a button. The performance for groups of 9, 99, 315, 513 and 999 kernel expansions were recorded and analysed. Frame time and number of frames per seconds were used to determine the performance of the application.

For the first version of asynchronous execution, tests were run by executing mini-stages from all expansions in a single frame. As seen in Figure 33, the mini-stages were poorly balanced. The execution time from certain stages took longer to execute. A quick and easy to implement solution was to delay the start of each expansion. This also increased the level of realism since, in reality, kernels do not pop all at the same time. As seen in Figure 34, there is a slight improvement for a low number of expansions, but the overall result is very unstable with a lot of spikes. For more than 315 kernels, the delay started to have a smaller influence since it could not compensate for the high frame time.

In the final version, the mini-stages were balanced better to run $1 - 2ms$. Compared to the previous version when running mini-stages from all expansions in a single frame (Figure 35),

the algorithm does not have spikes as big and finishes faster. The algorithm ends with a significant drop in frames per seconds since the engine allocates memory and initializes the new components (Animator, Skinned Mesh Renderer, etc.) of the expanded kernels.

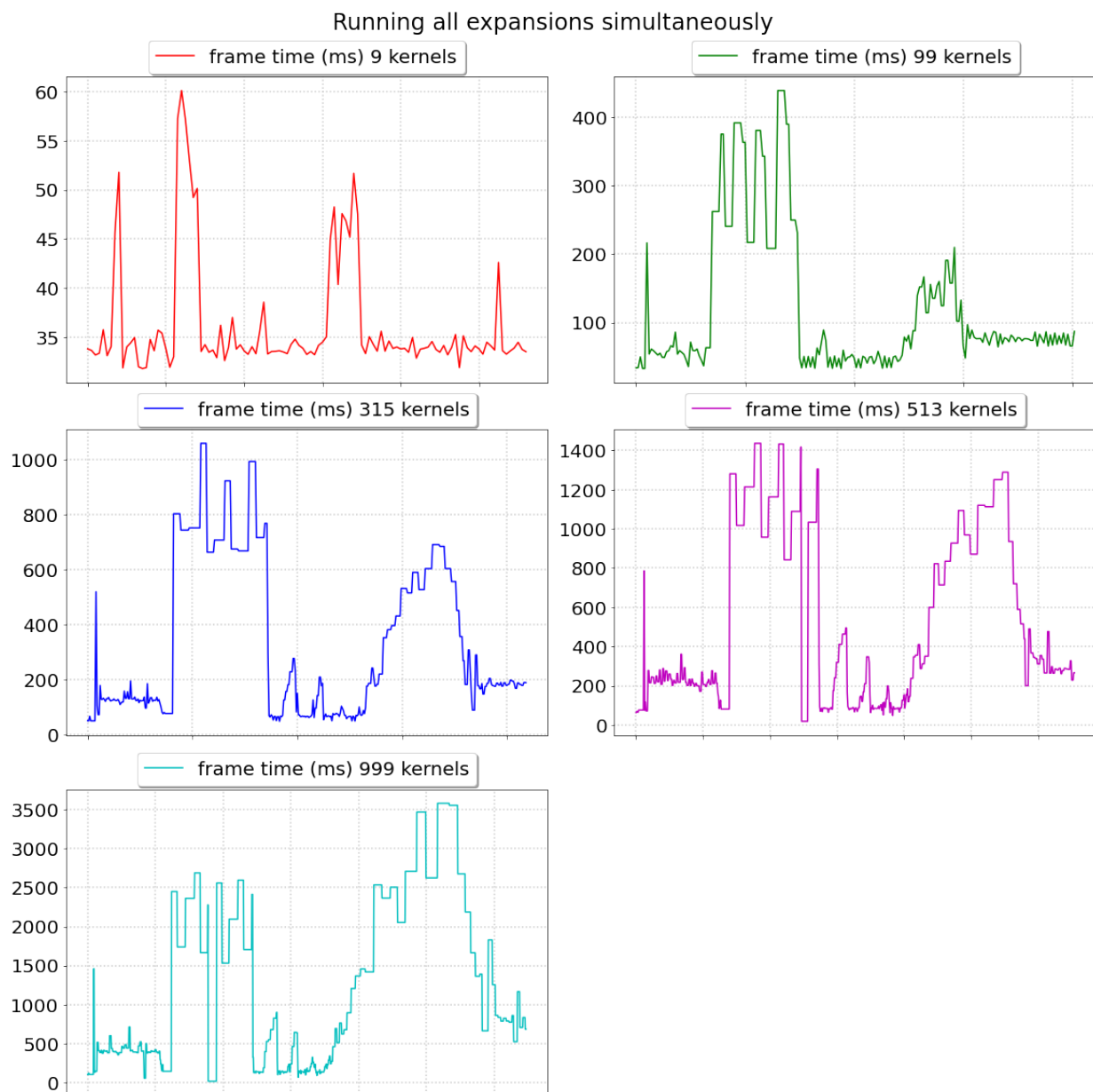


Figure 33: Frame time when executing mini-stages from all expansions in the same frame

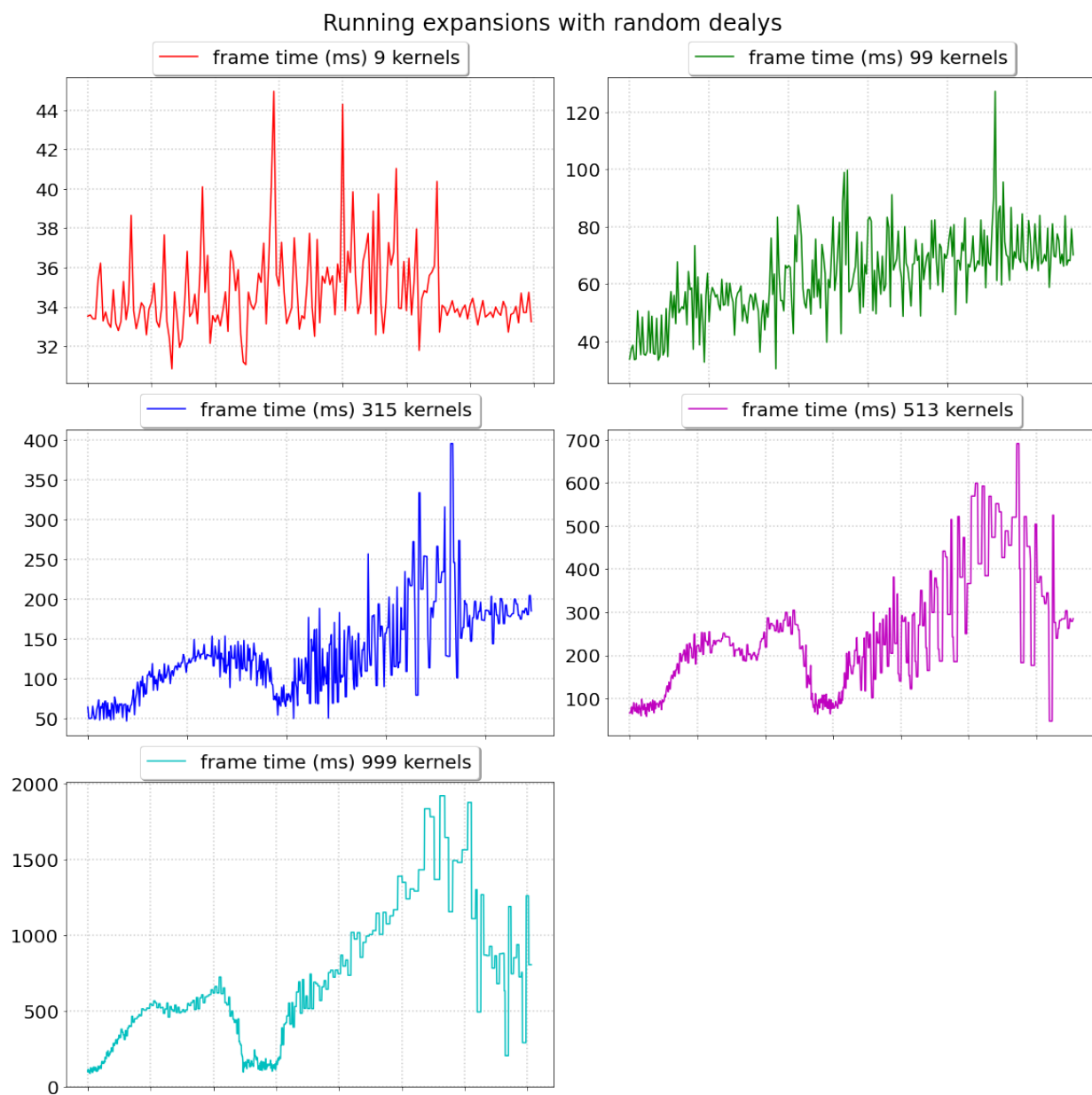


Figure 34: Frame time when executing mini-stages from all expansions in the same frame and the start-up is delayed

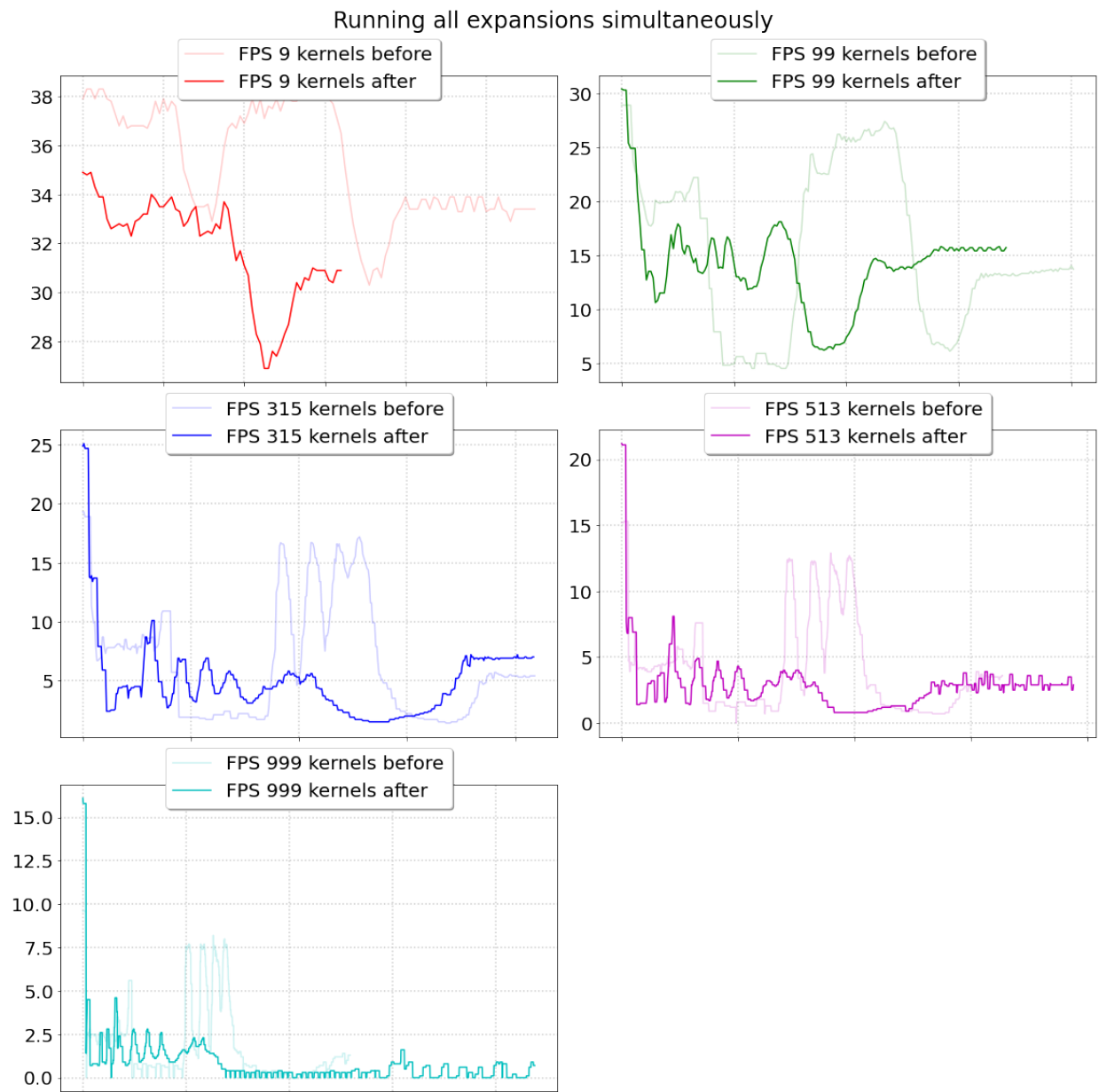


Figure 35: Comparison between the first and final versions of the algorithm

When analysing the Unity Profiler, the overhead can be seen more clearly (Figure 36). The blue part is the time spent in the Popcorn Generators scripts. It does not change significantly, but the time spent in Unity scripts (dark green) increases. Time spent calculating Physics (orange) and rendering (light green) also increases since the slices get collisions and the puff must also be rendered. In Figure 37, when expansions are complete, the time spent in the Physics module decreases since the rigid bodies of the expanded kernels stop moving and less calculation is done. Rendering take time since more meshes are needed to be drawn.

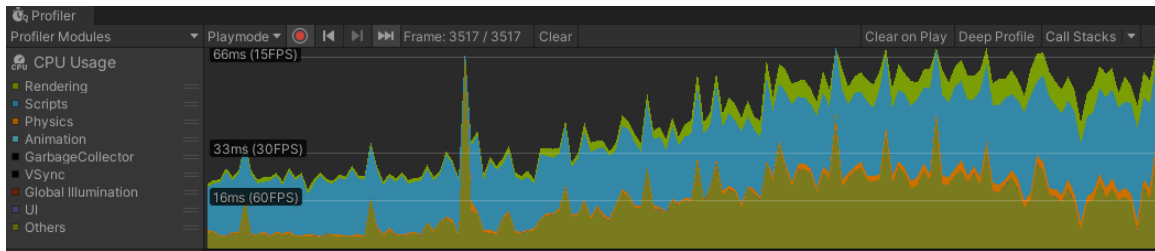


Figure 36: Unity Profiler when Unity Components are added to kernels (right)

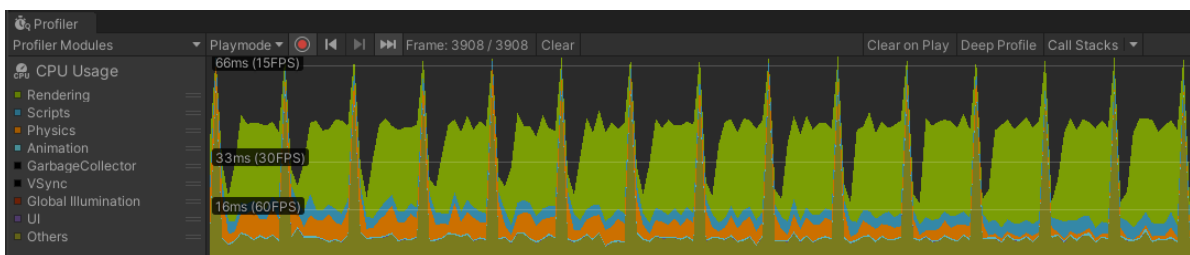


Figure 37: Unity Profiler when kernels stop moving

For further testing, a manager was created to manage the execution of expansions better. Instead of running the mini-stages of all expansions in a single frame, in each frame 17 expansions were advanced. The following frame processes the next 17 expansions and so on until all expansions are finished. This approach did not have a major influence to the time frame and frames per seconds. Even at 999 expansions, the frame rate is constant with less noise (Figure 38). Notably is the drop in frames per seconds when the expansions finish. The reason is the same as when expanding kernels simultaneously, overhead generated by the engine when calculating Physics and rendering.

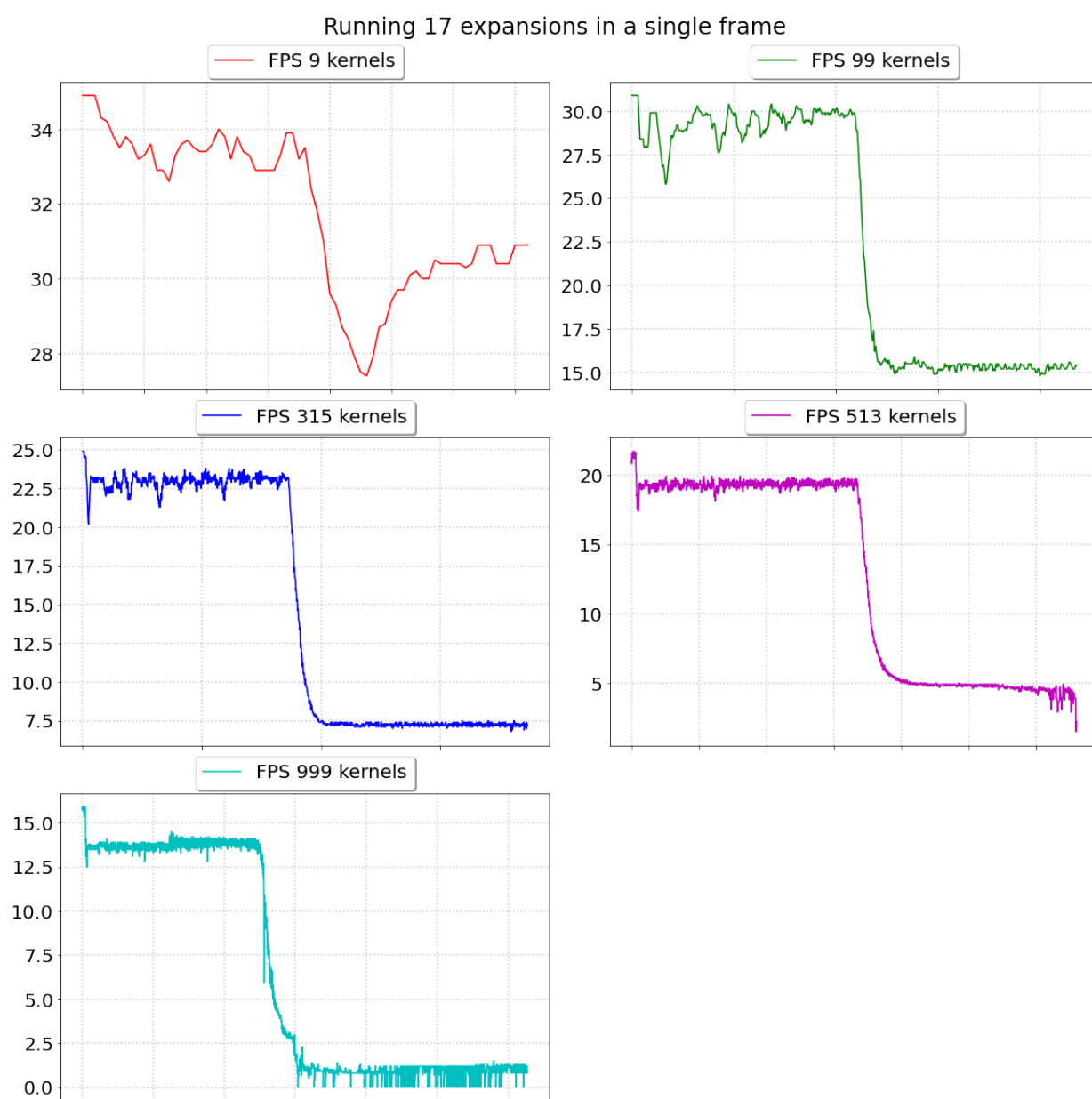


Figure 38: FPS when executing only 17 expansions each frame

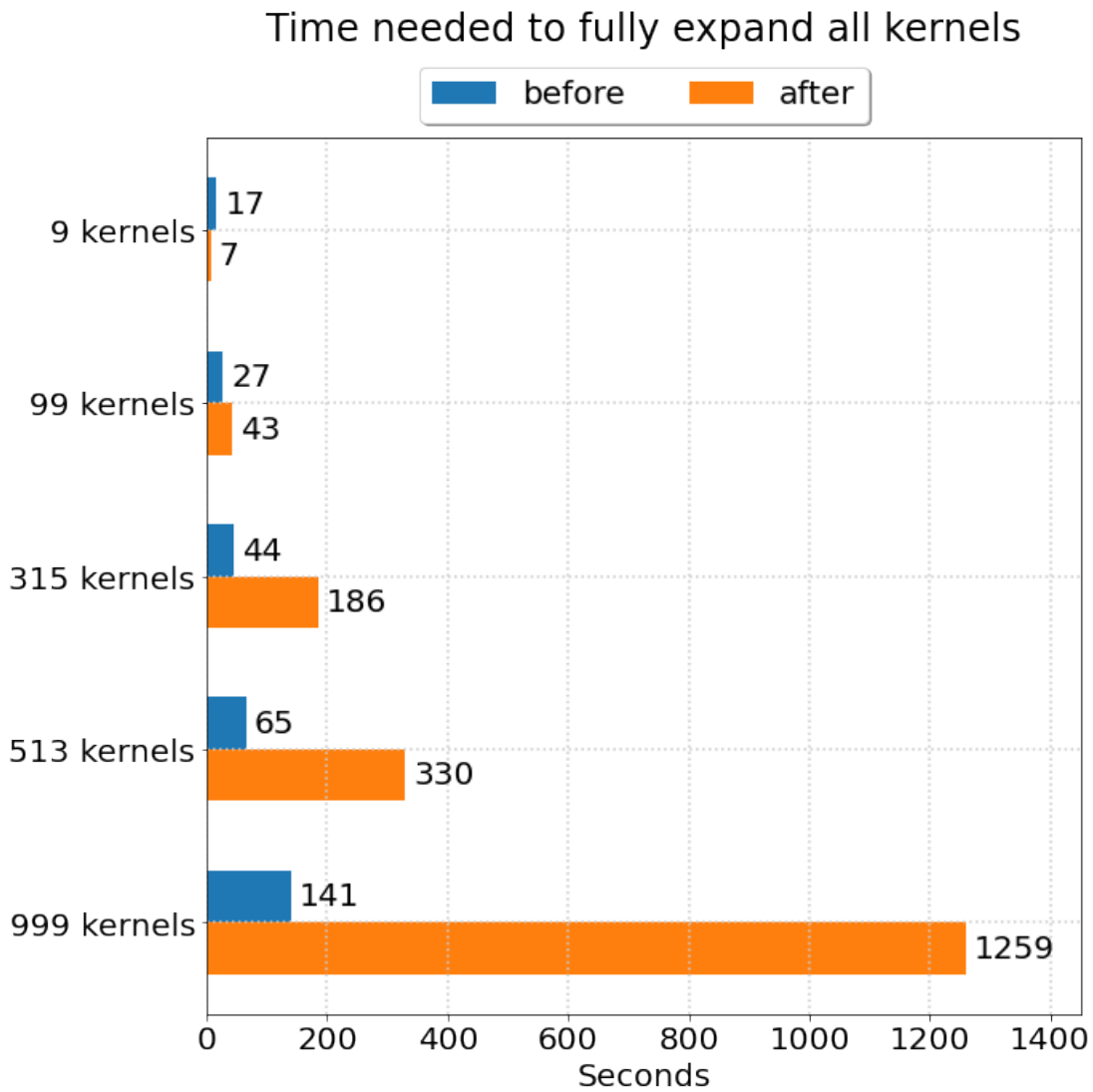


Figure 39: Before and after comparison of total time needed to expand all kernels in a scene

In the final version, the frames per second do not fluctuate very much. Compared to the previous versions, the stability of the expansion process has been improved. The biggest downside is that the simulation requires substantially more time to run and successfully expand all popcorn kernels (Figure 39). When expanding all kernels in the same frame, the newer implementation is faster, but, as number of kernels increases, it takes more frames to fully advance the expansions of all kernels. As seen in the figure, the last version takes 9 times more time to finish.

7 CONCLUSIONS

In this project, the expansion of popcorn kernel was studied and the natural phenomenon was fully understood. Several procedural generation techniques used in creating plants have been analysed. And an architecture which tries to recreate the whole process of expanding have been created and implemented in Unity Engine. During the development, several challenges have been met, such as poor performance, texture generation and bones weighting, but reasonable solutions have been implemented. A fast but not very accurate algorithm to generate UV coordinate was implemented. Different solutions to performance issues have been implemented and tested.

The current implementation is a good starting point for further research and development. Slicing could benefit some randomness to the cuts. UV coordinates could be generated by parametrisation algorithms¹. For a large number of kernels, a lot of colliders are added into the scene. The application could benefit from collisions calculated on the GPU. Multi-threading could also be an improvement since all execution is done on a single thread in the current prototype. And, the middle part could also be procedurally generated and not pre-generated.

¹https://en.wikipedia.org/wiki/Mesh_parameterization

ANNEX



Figure 40: Comparison between a real (right) and a generated (left) expanded kernel

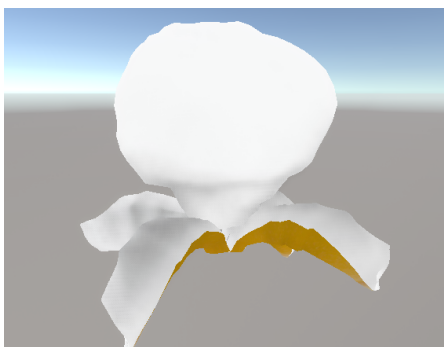


Figure 41: Untextured expanded kernel

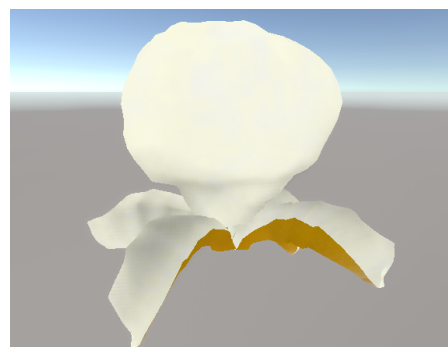


Figure 42: Textured expanded kernel.



Figure 43: Scene with a kitchen



Figure 44: Part of the kitchen where the kernels expand



Figure 45: Frying pan with generated expanded popcorn kernels

BIBLIOGRAPHY

- [1] Torsten Hädrich, Bedrich Benes, Oliver Deussen, and Sören Pirk. Interactive modeling and authoring of climbing plants. In *Computer Graphics Forum*, volume 36, pages 49–61. Wiley Online Library, 2017. Last accessed: 12 June 2022.
- [2] RC Hoseneý, K Zeleznak, and A Abdelrahman. Mechanism of popcorn popping. *Journal of cereal Science*, 1(1):43–52, 1983. Last accessed: 11 June 2022.
- [3] George Kelly and Hugh McCabe. A survey of procedural techniques for city generation. *ITB Journal*, 14(3):342–351, 2006. Last accessed: 11 June 2022.
- [4] Johan Knutzen, Suguru Saito, and Masayuki Nakajima. Generating climbing plants using l-systems. In *Proceedings of the Korean Society of Broadcast Engineers Conference*, pages 784–789. The Korean Institute of Broadcast and Media Engineers, 2009. Last accessed: 11 June 2022.
- [5] Emmanuel Viroť and Alexandre Ponomarenko. Popcorn: critical temperature, jump and sound. *Journal of the Royal Society Interface*, 12(104):20141247, 2015. Last accessed: 11 June 2022.